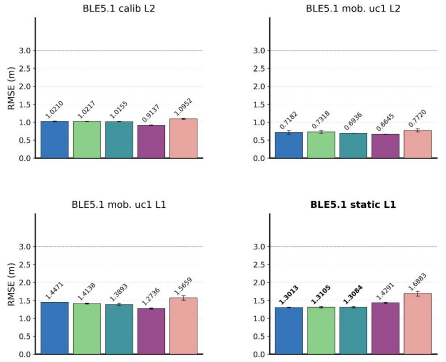
Training on Signal Processing and AI

Lê Trung Kiên - K69

Scientific computing with Python

1. NumPy — Tính toán số học

- Thư viện tính toán số học hiệu năng cao, sử dụng các thành phần được triển khai bằng C/C++ để tăng tốc xử lý.
- Dữ liệu được lưu trữ dưới dạng mảng đa chiều (`ndarray`), hỗ trợ biểu diễn và tính toán trên các tensor.
- NumPy hỗ trợ lưu trữ dữ liệu liên tiếp trong bộ nhớ (contiguous memory), giúp truy xuất dữ liệu hiệu quả hơn so với danh sách Python thông thường.
- Vectorization (vector hóa) là kỹ thuật thực hiện một phép toán trên toàn bộ mảng dữ liệu mà không cần viết vòng lặp cho từng phần tử trong Python, nhờ đó giảm overhead và tăng tốc tính toán.

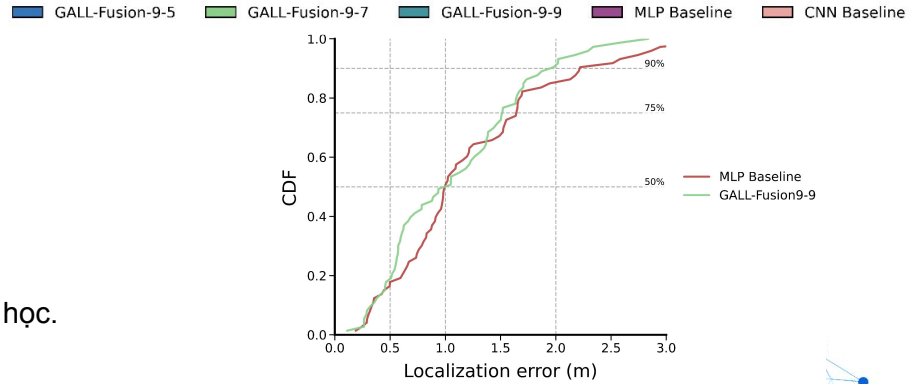


2. Pandas — Xử lý dữ liệu

- Đọc dữ liệu CSV dưới dạng bảng DataFrame.
- Hỗ trợ truy xuất, lọc và thao tác dữ liệu.
- Đơn giản hóa quá trình kiểm tra và tiền xử lý dữ liệu.

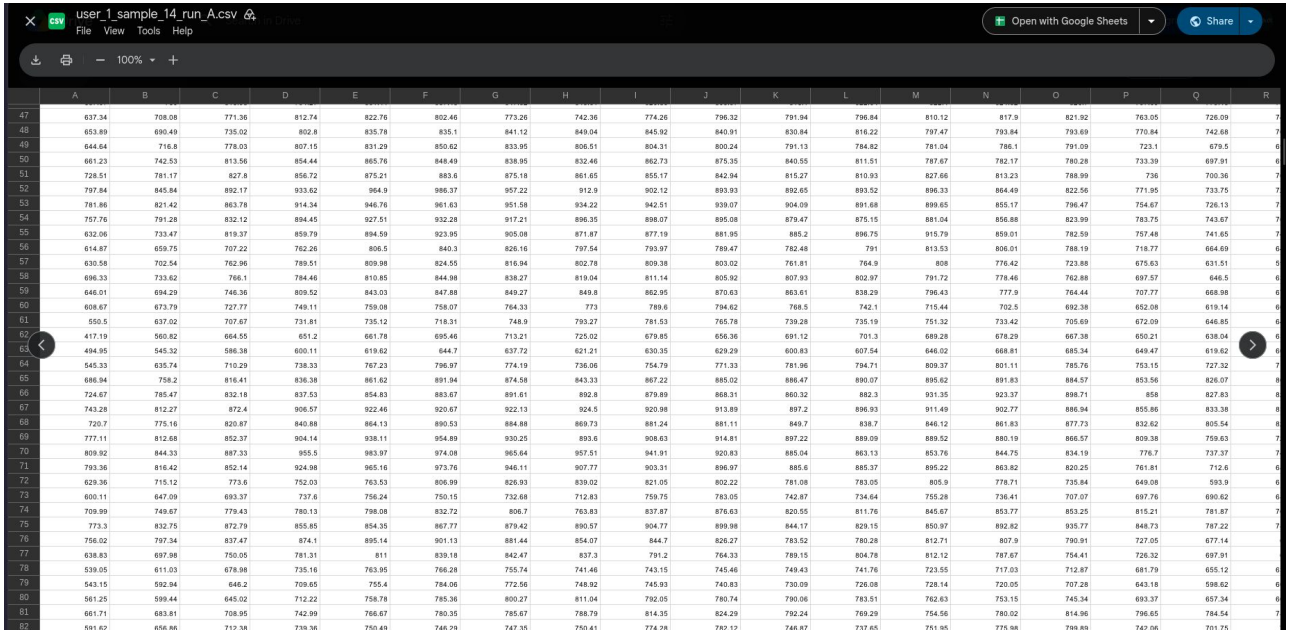
3. Matplotlib — Trực quan hóa dữ liệu

- Trực quan hóa tín hiệu, dữ liệu và kết quả tính toán.
- Hỗ trợ vẽ đồ thị đường, scatter plot và heatmap.
- Được sử dụng rộng rãi để tạo biểu đồ cho các bài báo khoa học.



Scientific computing with Python

Có 1 file csv nằm trong một bộ dataset CSI về nhận diện cử chỉ cơ thế người như sau: có 813 rows và 52 columns. 52 columns này tương đương với cả 52 sóng mang của wifi. Giá trị của mỗi cột là biên độ của từng sóng mang đấy. **Tần số lấy mẫu theo bài báo là 200Hz.**



Scientific computing with Python

Ta load file csv, lấy số hàng và số cột của file csv
Lưu ý là lúc load để file path nên địa chỉ tương đối thay vì mình nhiều lúc mình để terminal đang làm việc ở thư viện có chứa file .venv.

Dùng head() để lấy 5 rows đầu tiên

```python
import pandas as pd

df = pd.read_csv(os.path.join(os.getcwd(), "Signal_Processing", "csv_data.csv"))

# 2. Lấy về số hàng và số cột của DataFrame
n_rows, n_cols = df.shape

print("Number of rows:", n_rows)
print("Number of columns:", n_cols)

# 3. In ra header của file
print("\nColumn names:")
print(df.columns.tolist())

# 4. In 5 row đầu tiên
print("\nFirst 5 rows:")
print(df.head())
```

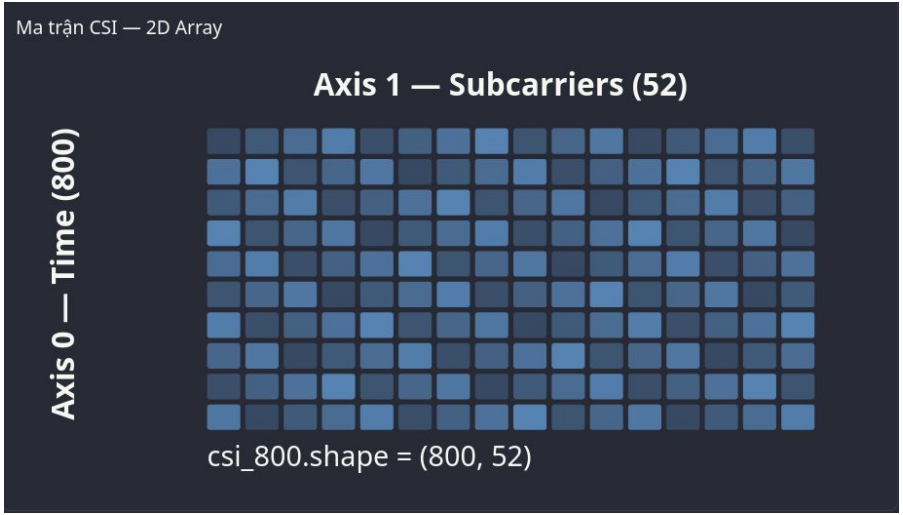
```
First 5 rows:
   375.03   376.4  385.63  417.89  432.26  429.32  430.67  433.22  ...  511.42   536.7     532  508.14   506.8  506.08  483.81  494.77
0  628.07  688.73  742.65  776.79  807.15  833.87  819.16  791.95  ...  872.43  936.84  936.14  893.84  892.75  891.93  845.13  854.98
1  668.07  697.22  730.88  777.77  805.10  813.52  798.24  776.54  ...  798.54  828.58  798.82  730.85  763.58  797.20  720.13  725.97
2  730.76  770.20  807.90  840.51  870.74  898.69  883.38  855.03  ...  761.33  839.87  830.18  764.14  758.84  757.42  713.89  743.97
3  611.15  690.29  758.69  795.58  821.61  837.13  800.09  748.21  ...  620.28  725.90  740.85  697.88  673.05  647.66  597.01  643.08
4  693.94  733.65  775.92  825.72  854.45  862.80  841.38  810.99  ...  712.70  757.14  784.30  800.40  810.62  807.62  713.34  715.85

[5 rows x 52 columns]
```

Câu hỏi: Làm thế nào để biết thời lượng của tín hiệu là bao nhiêu?

Vì ta biết tần số lấy mẫu là 200Hz, nên ta có thể lấy tổng số mẫu (tương đương 813 rows) chia 200 để biết rằng thời lượng của tín hiệu là 4.065s
→ Về sau sẽ cần chuẩn hóa về cùng một thời lượng duration cố định nếu train AI

```
fs = 200
duration = len(df) / fs # 4.065
```

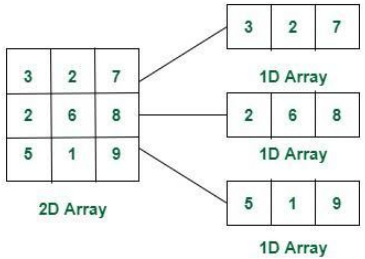


Scientific computing with Python

Ta có 2 cách để gọi hàm loc() và iloc() để lấy dữ liệu cần theo hàng hoặc cột. loc() sử dụng index (số chỉ) lấy dữ liệu (*slicing operator*)

- Công thức chung cho slicing operator là “start:end:step”, index bắt đầu, index kết thúc và bước đi (mặc định là 1)
- “:” để không tức là quét hết cả array đấy
- Mặc định: range(n) là “0;n” hay lấy từ 0 đến n -1

```
29
30  df.columns = [f"SC_{i+1}" for i in range(df.shape[1])] # Gán tên cột cho dễ gọi
31  print(df.head())
32  # Giá trị biên độ tất cả subcarrier của packet 1
33  packet_1 = df.iloc[0, :]
34  # Subcarrier đầu tiên của tất cả packet
35  sc_1 = df.iloc[:, 0]
36  # 100 packet đầu tiên, 10 subcarrier đầu tiên
37  subset = df.iloc[0:100, 0:10]
38  print("subset shape:", subset.shape)
39  # Chọn subcarrier 1 theo tên cột
40  sc_1 = df.loc[:, "SC_1"]
41  # chọn nhiều subcarrier theo tên cột (trên thực tế cũng có thể làm được bằng cách dùng iloc
42  selected = df.loc[:, ["SC_1", "SC_5", "SC_10"]]
43  # Chọn 100 packet đầu tiên, 10 subcarrier đầu tiên theo tên cột
44  subset = df.loc[0:99, "SC_1":"SC_10"]
45
46  print(selected.shape)  # (813, 3)
47  print(subset.shape)    # (100, 10)
```




```
[5 rows x 52 columns]
     SC_1    SC_2    SC_3    SC_4    SC_5    SC_6    SC_7    SC_8  ...   SC_45   SC_46   SC_47
0  628.07  688.73  742.65  776.79  807.15  833.87  819.16  791.95  ...  872.43  936.84  936.14
1  668.07  697.22  730.88  777.77  805.10  813.52  798.24  776.54  ...  798.54  828.58  798.82
2  730.76  770.20  807.90  840.51  870.74  898.69  883.38  855.03  ...  761.33  839.87  830.18
3  611.15  690.29  758.69  795.58  821.61  837.13  800.09  748.21  ...  620.28  725.90  740.85
4  693.94  733.65  775.92  825.72  854.45  862.80  841.38  810.99  ...  712.70  757.14  784.30

[5 rows x 52 columns]
subset shape: (100, 10)
(813, 3)
(100, 10)
```

Scientific computing with Python

Ta có thể load Data Frame về numpy rồi xử lý tiếp

Câu hỏi: Các lệnh bên phải thực hiện chức năng gì?

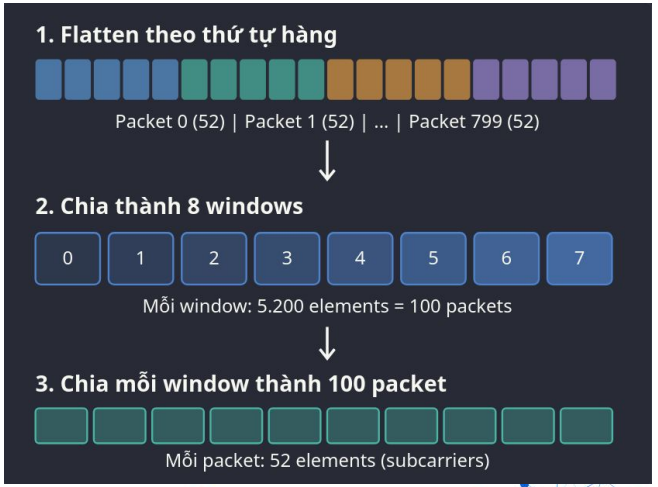
```python
csi = df.to_numpy(dtype=np.float64)

segment = csi[200:400, :]
selected = csi[:, [0, 5, 10, 15]]
```

Lệnh reshape: có để biến đổi về dạng tensor có n chiều → tổ chức dữ liệu theo batch, cho phép mô hình xử lý nhiều mẫu cùng lúc, → ttính toán song song trên GPU (các mẫu trong batch được xử lý song song).

800 packets được chia thành 8 windows, mỗi window có 100 packets × 52 subcarriers.

```python
# Chọn 800 packet đầu tiên, tất cả subcarrier
csi_800 = csi[:800, :]
# Chia thành 8 window, mỗi window có 100 packet, tất cả subcarrier
#  --> Cấu trúc tensor 3 chiều (8, 100, 52)
windows = csi_800.reshape(8, 100, 52)
```



Câu hỏi: Nếu gọi reshape(400,104) và (400,106) thì cái nào lỗi?

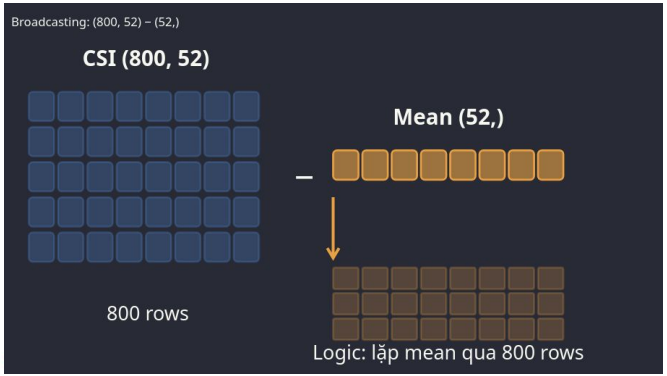
Scientific computing with Python

Vectorization: thực hiện một phép toán trên toàn bộ mảng một cách ***song song*** thay vì phải thực hiện vòng lặp tuần tự.




```
'''
Vectorization example: multiply all elements of csi by 2 and store in result
'''
result = np.empty_like(csi)
# Normal method:
for i in range(800):
    for j in range(52):
        result[i, j] = csi[i, j] * 2
# Vectorize:
result = csi * 2
```

Broadcasting: thực hiện phép toán trên tensor với các ***shape khác nhau*** nhưng mà framework tự chỉnh shape để tương thích logic




```
'''
Broadcasting example
'''
# Với từng subcarrier, tính mean và std của tất cả packet
mean = np.mean(csi, axis=0)
std = np.std(csi, axis=0)

print(mean.shape)  # (52,)
print(std.shape)   # (52,)

# Chuẩn hóa dữ liệu: với từng subcarrier, trừ đi mean và chia cho std (Z-score)
csi_normalized = (csi - mean) / (std + 1e-8)

print(csi_normalized.shape)  # (813, 52)

result = np.empty_like(csi)
```

Scientific computing with Python

Figure: 1 hình tổng thể
Axe: 1 vùng / subplot của figure

- Title & Labels: Tiêu đề và nhãn các trục, giúp mô tả nội dung và đơn vị của dữ liệu.
- Ticks: Các vạch chia và giá trị hiển thị trên trục tọa độ, giúp xác định vị trí dữ liệu.
- Legend: Chú thích để phân biệt các đường tín hiệu hoặc nhóm dữ liệu trên cùng biểu đồ.
- Annotation: Bổ sung ghi chú, mũi tên hoặc đánh dấu những điểm dữ liệu quan trọng.
- Style: Tùy chỉnh màu sắc, kiểu đường, độ dày nét, font chữ và grid để tăng tính trực quan.

```python
def plot_csi_subcarriers(df, fs=200):
    # Create 2x2 subplots
    fig, axes = plt.subplots(2, 2, figsize=(10, 6))

    # Time axis
    time = np.arange(len(df)) / fs

    # Select 4 subcarriers
    subcarriers = ["SC_1", "SC_10", "SC_20", "SC_30"]

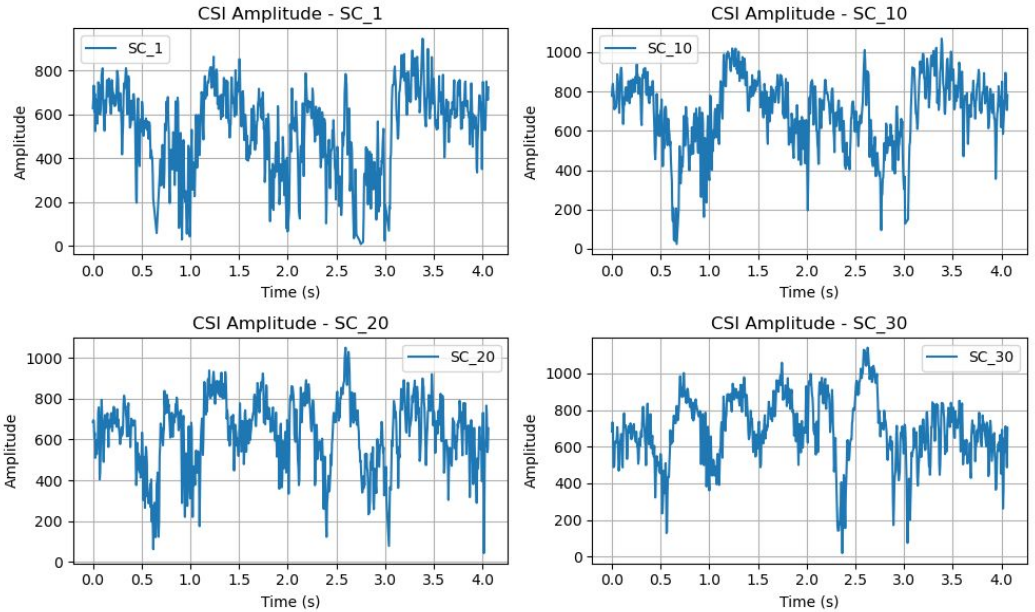
    for ax, sc in zip(axes.flat, subcarriers):
        ax.plot(time, df[sc], label=sc)

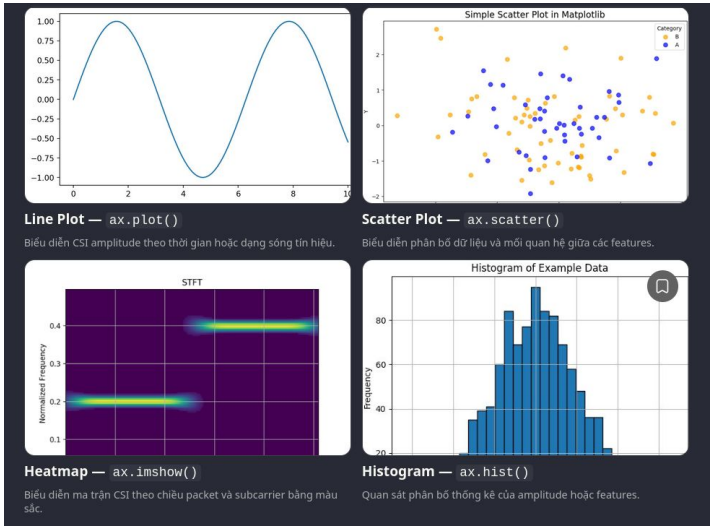
        ax.set_title(f"CSI Amplitude - {sc}")
        ax.set_xlabel("Time (s)")
        ax.set_ylabel("Amplitude")
        ax.legend()
        ax.grid(True)

    fig.tight_layout()
    plt.savefig(os.path.join("Signal_Processing", "csi_subcarriers.png"))

plot_csi_subcarriers(df)
```

Scientific computing with Python

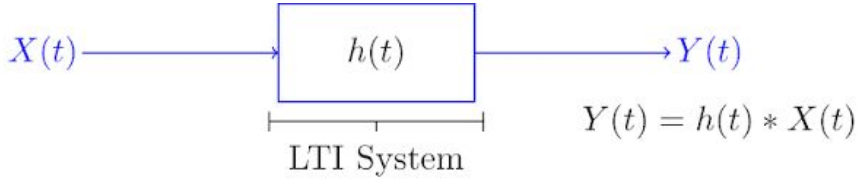




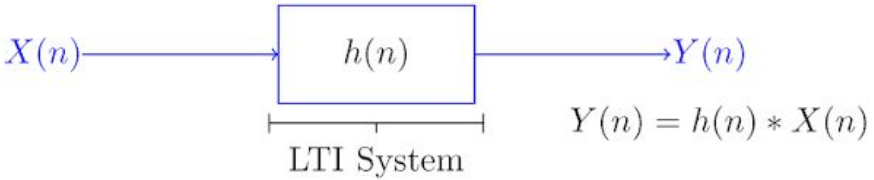
Nên tham khảo plot: https://github.com/ChenLiu-1996/figures4papers

Signal processing: Convolution, correlation & LTI system

Unit impulse δ[n]: Tín hiệu chỉ bằng 1 tại n=0, bằng 0 tại những vị trí khác.
Impulse response (h[n]: Đầu ra của hệ thống LTI khi đầu vào là δ[n]
Với hệ thống LTI, *đáp ứng xung mô tả đầy đủ hành vi vào–ra của hệ thống.*



n: Xác định vị trí của mẫu đầu ra (y[n] mà ta đang tính.
k: Chạy qua từng mẫu đầu vào (x[k], từ đó tính đóng góp của mẫu này vào đầu ra tại n.
x[k]: Biên độ của mẫu đầu vào tại vị trí k.
h[n-k]: Giá trị đáp ứng xung tại thời điểm n, gây ra bởi một xung được đưa vào hệ thống tại thời điểm k.



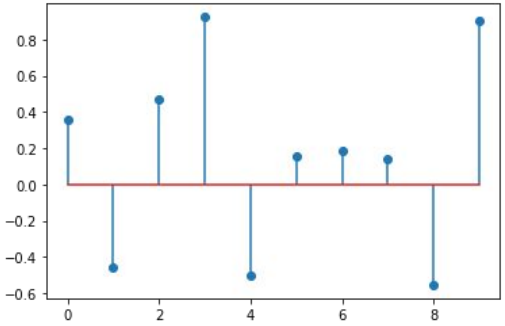
$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

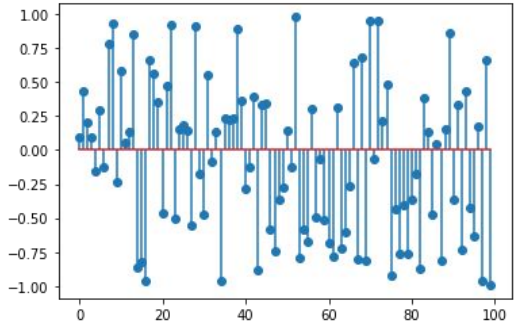
Signal processing: Convolution, correlation & LTI system

Cross-correlation: Đo mức độ tương đồng giữa hai tín hiệu khi dịch tương đối theo thời gian, hữu ích để tìm độ trễ.

VD: Tín hiệu bên trái (h) là index 19:29 của tín hiệu phải (x)

Khi ta thực hiện np.correlate(x,h) thì vị trí đỉnh là 19 tại vì tại điểm này tín hiệu có độ tương đồng cao nhất

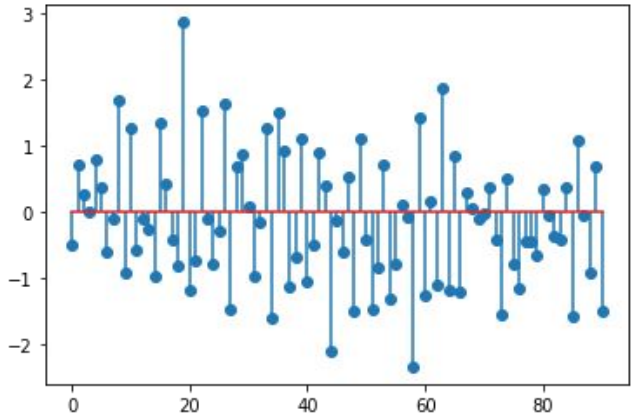




$$r_{xy}(l) = \sum_{n=-\infty}^{\infty} x(n)\ y(n-l)$$

OR

$$r_{xy}(l) = \sum_{n=-\infty}^{\infty} x(n+l)\ y(n)$$

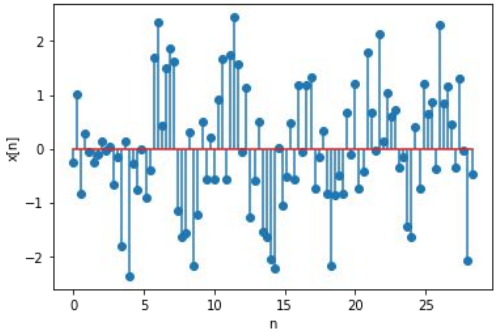


Signal processing: Convolution, correlation & LTI system

Autocorrelation: Đo sự tương đồng của tín hiệu với chính nó tại các độ trễ khác nhau, hỗ trợ phát hiện tính tuần hoàn.

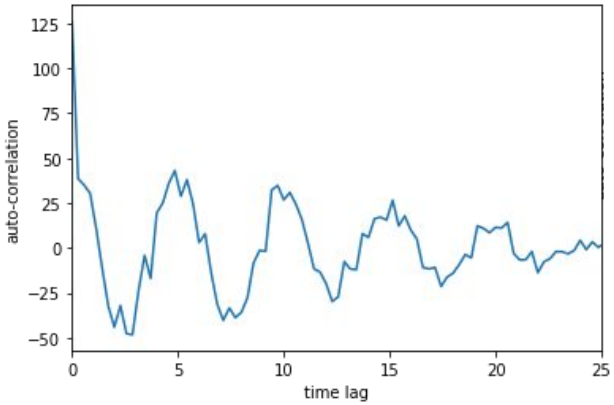
VD: Tín hiệu là tín hiệu sin(2pi/5 n) + nhiễu

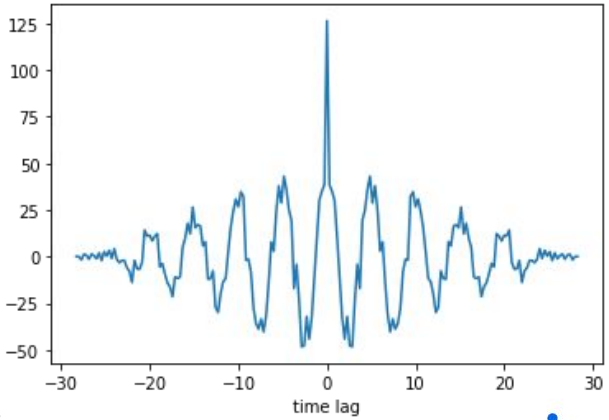
Khi ta thực hiện np.correlate(x,x) thì các peak cách có xu hướng cách nhau 5 mẫu → cứ 5 mẫu có độ tương quan cao → chu kì nó là 5



$$r_{x,x}(l) = \sum_{n=-\infty}^{+\infty} x(n)x(n-l)$$

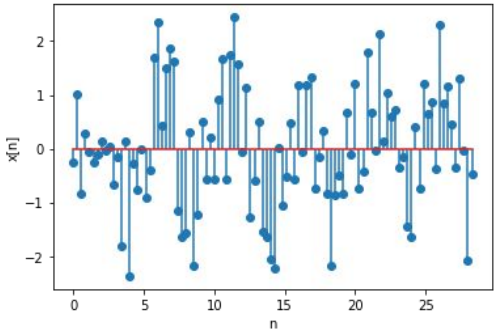
Câu hỏi: Mạng convolution neural network thực ra thường dùng phép correlation chứ không phải convolution, vì sao?





Signal processing: Signal as a vector & Orthogonal basis

Tín hiệu rời rạc hữu hạn gồm N mẫu có thể được biểu diễn thành một vector trong không gian N chiều.
Orthogonal Basis: Các vector cơ sở đôi một trực giao, giúp tính các hệ số biểu diễn tín hiệu một cách đơn giản. Trên miền thời gian, coi mỗi điểm như một vector cơ sở nhân lên 1 hệ số VD: a_1 * [0,1,0,0…]



Các hàm mũ phức tại những tần số DFT khác nhau tạo thành một hệ cơ sở trực giao trên N mẫu.

Fourier Series (FS): Biểu diễn tín hiệu liên tục, tuần hoàn thành tổng các hàm sin/cos ở những tần số điều hòa rời rạc.
Fourier Transform (FT): Phân tích tín hiệu liên tục theo thời gian, không nhất thiết tuần hoàn, thành các thành phần tần số liên tục.
Z-Transform: Biểu diễn tín hiệu **rời rạc** bằng các hàm cơ sở z^(-n), với z = $r.e^{jw}$. Khi r=1, ta xét trên đường tròn đơn vị và thu được DTFT nếu điều kiện hội tụ thỏa mãn. Nó **không** dựa trên cơ sở trực giao.
Discrete Fourier Transform (DFT): Phân tích chuỗi hữu hạn N mẫu thành **N thành phần tần số rời rạc**, thuận tiện cho tính toán trên máy tính.

Cơ sở Fourier:

$$u_k[n] = e^{j2\pi kn/N}$$

Tính trực giao:

$$\sum_{n=0}^{N-1} u_k[n]\overline{u_m[n]} = \begin{cases} N, & k = m \\ 0, & k \neq m \end{cases}$$

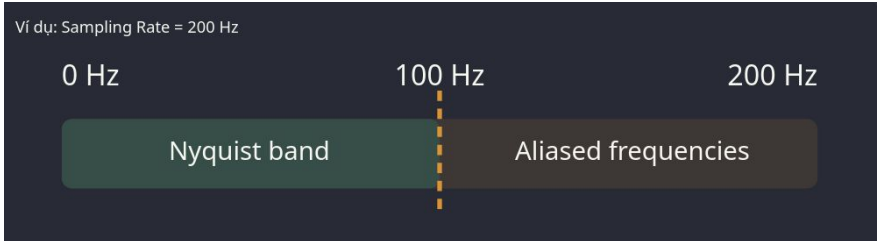
Đây là lý do ta có thể tách những thành phần tần số khác nhau bằng phép chiếu.

Signal processing: properties

Tín chất dịch thời gian và dịch tần số.
Ứng dụng: dịch baseband lên passband trước khi cho qua kênh vô tuyến.

Time Shifting	$x[n-n_0] \leftrightarrow e^{-j\omega n_0} X(e^{j\omega})$
Frequency Shifting	$e^{j\omega_0 n} x[n] \leftrightarrow X(e^{j(\omega-\omega_0)})$

Tích chập trên miền thời gian là phép nhân trên miền tần số.
Ứng dụng: tính nhanh y[n]
pilot subcarrier cho X[k] cố định ko phụ thuộc data, lấy Y[k] / X[k] = H[k] tương ứng với cả đáp ứng kênh của kênh truyền

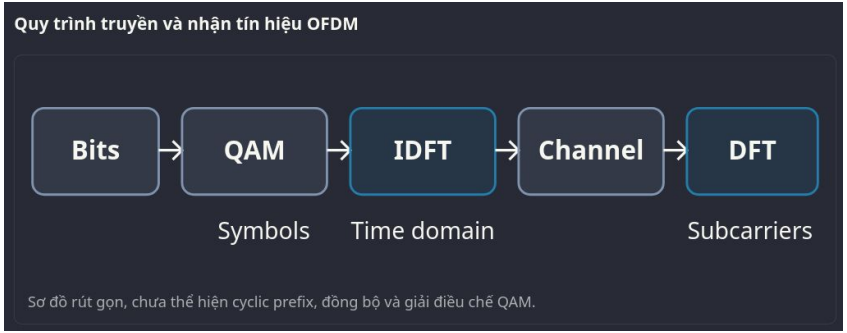


Nyquist Sampling Theorem: Với tín hiệu giới hạn băng ở f_max, cần sampling rate lớn hơn 2*f_max để tránh aliasing theo điều kiện lấy mẫu thông thường.



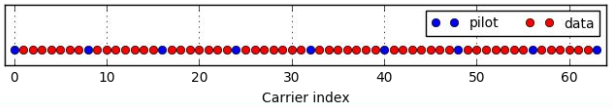
Signal processing: Basic of OFDM

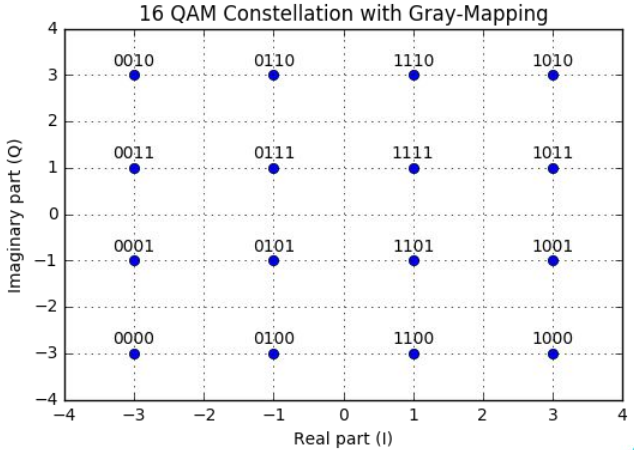
Bước 1 — QAM Modulation: Ở phía phát, chuỗi bit được ánh xạ thành các symbol QAM có biên độ và pha khác nhau, sau đó phân bổ lên các subcarrier (tương ứng hàm mũ phức và dễ xảy ra chồng phổ)
Bước 2 — IDFT/IFFT: Tổng hợp các symbol/subcarrier trực giao để tạo thành một tín hiệu **OFDM trong miền thời gian**.



Ý nghĩa của tính trực giao: Các subcarrier có thể **chồng lấn phổ** nhưng vẫn được phân tách nhờ phép chiếu DFT, giúp sử dụng băng thông hiệu quả và hạn chế nhiễu giữa các subcarrier.

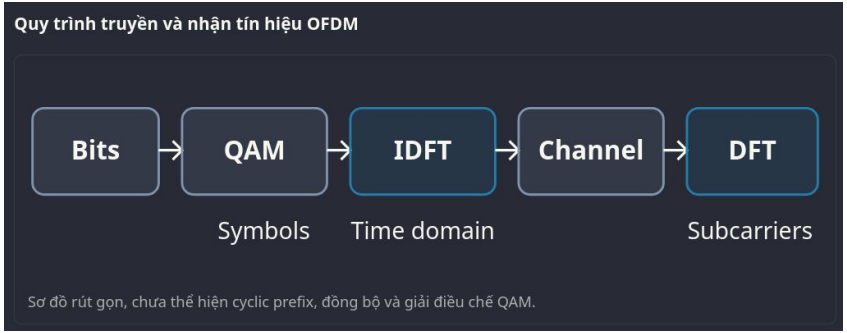
$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] u_k[n]$$





Signal processing: Basic of OFDM

Bước 3 — Transmission
Bước 4 — DFT/FFT: Ở phía thu, sau các bước xử lý cần thiết, DFT phân tích tín hiệu nhận theo các cơ sở trực giao để tách tín hiệu trên từng subcarrier.
Bước 5 — QAM Demodulation: Sau khi ước lượng và bù ảnh hưởng kênh truyền, các symbol được giải điều chế để khôi phục chuỗi bit ban đầu.



2. CSI Sensing thực sự quan sát điều gì?

Khi con người di chuyển, đường truyền phản xạ thay đổi, dẫn đến đáp ứng kênh $H[k]$ thay đổi.

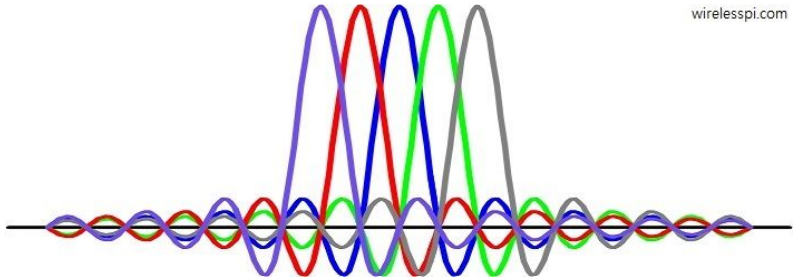
Nếu thu được nhiều WiFi packets liên tiếp, ta có:

$$H[k,t]$$

Trong đó k là chỉ số subcarrier và t là thời gian quan sát.

Từ đó, có thể theo dõi:

- **CSI Amplitude:** $|H[k,t]|$, thể hiện độ lớn đáp ứng kênh.
- **CSI Phase:** $\angle H[k,t]$, thể hiện pha của đáp ứng kênh.



Ox là miền tần số

Deep learning fundamentals

Hàm loss function đo sự khác biệt giữa output và target ban đầu và mục tiêu là tối thiểu hóa sự khác biệt giữa output và target.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Loss functions: MSE, MAE (regression), Cross-entropy (classification)

$$CE = -\frac{1}{N}\Sigma_{i=1}^{N}\Sigma_{j=1}^{C}(y_{i,j}.log(p_{i,j}))$$

Trong lý thuyết thông tin thì entropy cho số bit tối thiểu để mã hóa một lượng thông tin tuân theo phân phối xác xuất nào đó. Cross-Entropy biểu diễn số lượng bits cần dùng để mã hóa thông tin của target dùng phân phối dự đoán của model. Vì thế thì số lượng bits càng nhỏ thì sự sai khác giữa phân phối của dự đoán và của thực tế là nhỏ nhất.

Gradient Descent là thuật toán dùng để tìm cực tiểu của một hàm số, thường là hàm mất mát (loss function) trong quá trình huấn luyện mô hình.

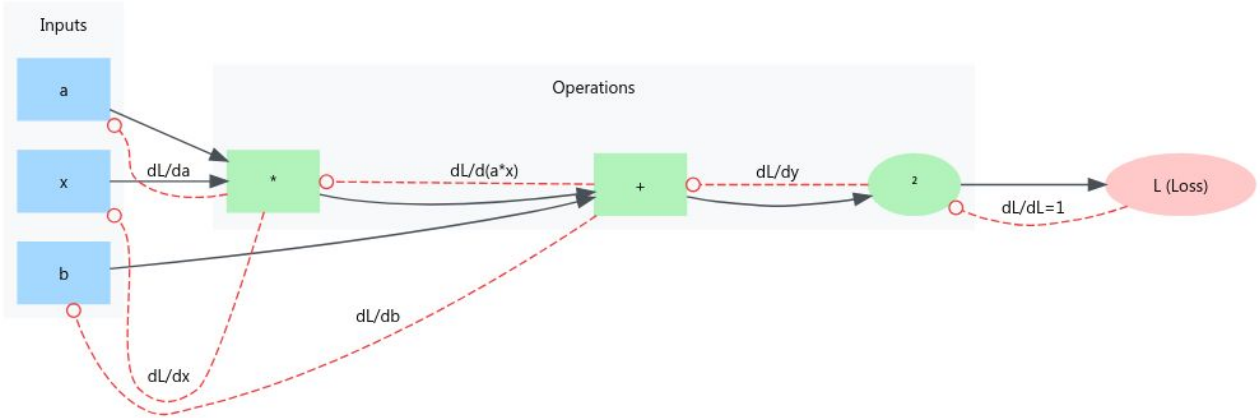
$$\mathbf{w}_{new} = \mathbf{w}_{old} - \alpha \cdot \nabla L(\mathbf{w})$$

Gradient là **vector chứa tất cả các đạo hàm riêng.** Hãy tưởng tượng bạn đang đứng trên một ngọn đồi và muốn đi xuống đáy → i ngược hướng hướng dốc nhất là ngược hướng gradient

Deep learning fundamentals

Với pytorch thì khi tạo ra tensor (vector n chiều) có thiết lập requires_grad=True là tạo gradient để nó lưu gradient vào graph mà để về sau tính đạo hàm.Bước tính cuối cùng là hàm loss và từ đây dùng backward() để truy ngược lại đạo hàm.

Note : backward chỉ gọi cho scalar (1 giá trị đơn nhất). Nếu không thì ta phải định nghĩa cách tính grad riêng. VD như f(x) = (y1, y2) thì tính đạo hàm không biết là d(y1)/dx hay d(y2)/dx

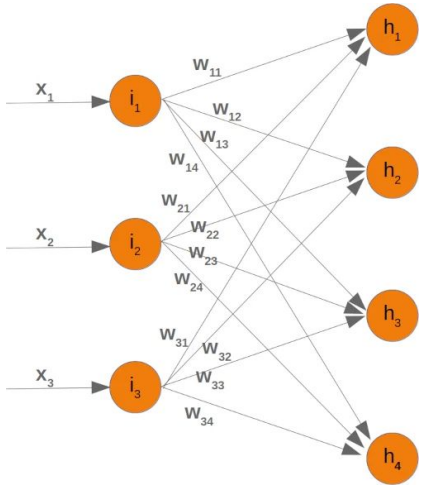


Deep learning fundamentals

VD:

y = w*x + b; Loss: L = y^2

Grad: L(x) = 2*y*w




```python
import torch
# Example setup (imagine these are results from a model)
x = torch.tensor([2.0,1.0], requires_grad=True)
w = torch.tensor([3.0,4.0], requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)

# Perform some operations (building the graph)
y = w * x + b # y = [3*2 + 1, 4*1 + 1] = [7, 5]
loss = y * y # loss giả làm ví dụ
loss=loss.sum() # Tổng các thành phần loss để có một scalar

print(f"Loss: {loss.item()}") # Loss: 74.0 (7^2 + 5^2 = 49 + 25 = 74)
# Compute gradients
loss.backward()
# After backward pass, gradients are populated
print(f"Gradient for x after backward: {x.grad}") # d(loss)/dx = d(y^2)/dx = 2*y*
(dy/dx) = 2*y*w = 2*[7,5]*[3,4] = [42,40]
print(f"Gradient for w after backward: {w.grad}") # d(loss)/dw = d(y^2)/dw = 2*y*
(dy/dw) = 2*y*x = 2*[7,5]*[2,1] = [28,10]
print(f"Gradient for b after backward: {b.grad}") # d(loss)/db = Σd(y^2)/db =
Σ2*y*(dy/db) = Σ2*y*1 = 2*(7+5) = 24
```

```
Loss: 74.0
Gradient for x after backward: tensor([42., 40.])
Gradient for w after backward: tensor([28., 10.])
Gradient for b after backward: 24.0
```

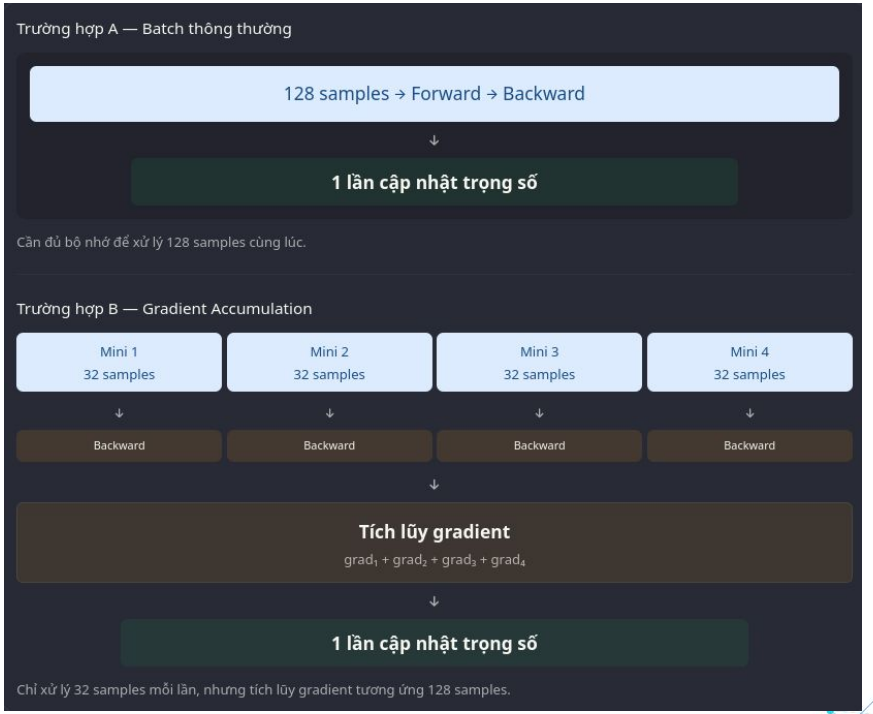
Deep learning fundamentals

Trong tình huống thực tế ta chia thành các batch. Nếu tensor lớn, một lần backward sẽ phải đi lại dữ liệu rất lớn, tiêu nhiều bộ nhớ. Thay vì đó ta chia thành các mini-batch size nhỏ với kích cỡ B cho mỗi batch → Mini Batch Stochastic Gradient Descent:

kết quả được tính cho mỗi batch để rồi cộng dồn lại

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}_t} \partial_{(\mathbf{w},b)} l^{(i)}(\mathbf{w}, b).$$

Ký hiệu	Ý nghĩa
$\mathbf{w}$	Vector trọng số (weights) của mô hình
b	Bias của mô hình
η	Learning rate – tốc độ học
$\mathcal{B}_t$	Tập các mẫu dữ liệu trong mini-batch tại bước t
$\|\mathcal{B}\|$	Số lượng mẫu trong mini-batch (batch size)



Deep learning fundamentals

Linear Layer thực hiện phép biến đổi tuyến tính theo cách gọi thông dụng trong Deep Learning (affine transformation)

Nếu chỉ xếp chồng nhiều Linear Layer mà không có Activation Function, toàn bộ mạng vẫn có thể biểu diễn bằng một phép biến đổi affine duy nhất. Activation Function *để đưa tính phi tuyến (non-linearity) vào mô hình.*

Note: activation function phải tính được đạo hàm

Input ban đầu có size

```
[
[x0,x1,..., x19],
...
(tổng cộng 64 vector hay kích cỡ batch là 64)
...
[x0,x1,..., x19]
]
```

Ở ví dụ trên thì shape được tính như sau

(64, 20) * (20, 30) + (30) = (64,30)

Thường là lớp cuối trong các model CNN, RNN,...

Mô phỏng $y = x.W^T + b$

Đây cũng được gọi khác là fully connected layer / dense layer

Python

```python
import torch
import torch.nn as nn

# Example: Expecting input features of size 20, producing output features of
size 30
linear_layer = nn.Linear(in_features=20, out_features=30)
# Create a sample input tensor (batch size 64, 20 features)
input_tensor = torch.randn(64, 20)
# Pass the input through the layer
output_tensor = linear_layer(input_tensor)

print(f"Input shape: {input_tensor.shape}")
print(f"Output shape: {output_tensor.shape}")
# Inspect the layer's parameters (automatically created)
print(f"\nWeight shape: {linear_layer.weight.shape}")
print(f"Bias shape: {linear_layer.bias.shape}")
```

Deep learning fundamentals

Convolutional Layer khai thác cấu trúc không gian cục bộ (local spatial structure) thông qua các kernel, trong khi Linear Layer không có cơ chế khai thác quan hệ không gian một cách tường minh.

Trong ảnh ví dụ

Input: 5×5

Padding: 2 (vị trí pad thêm vào có giá trị là 0)

Kernel: 4×4

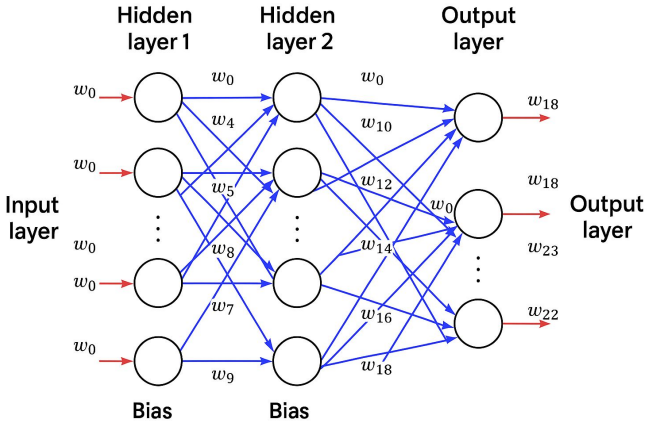
Stride: 1

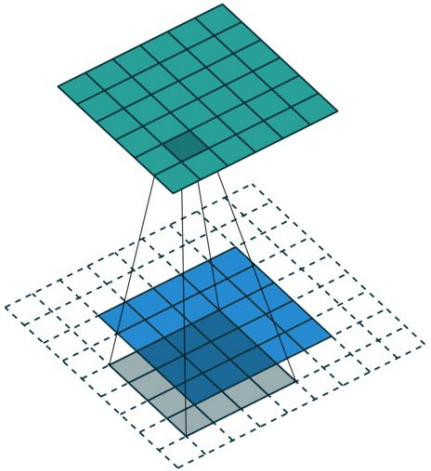
Output: 6×6

The Formula

For a square input of size $W \times W$, a filter of size $K \times K$, padding P, and stride S, the output size O is: Stack Overflow +4

$$\text{Output Size } (O) = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$





Deep learning fundamentals

